Replit 3.0 Prompt - Unicorn Systemuppdateringar

KONTEXT

Detta dokument beskriver de nya funktionerna som ska implementeras i systemet för Kinab. Ändringarna baseras på en uppdaterad systemspecifikation som introducerar avancerad metadatahantering för både objekt och uppgifter/ordrar.

Basverksamhet (Kinab):
Kinab är ett serviceföretag som hanterar sophämtning, containerhantering och underhåll av avfallsbehållare för bostadsrättsföreningar och fastighetsbolag. Systemet Modus 2.0 används för att planera, organisera och utföra dessa tjänster genom en klusterbaserad organisationsmodell där objekt (t.ex. matavfallsbehållare, återvinningsstationer) organiseras hierarkiskt och kopplas till kunder, platser, priser och uppgifter.

DEL 1: SAMMANFATTNING AV ÄNDRINGAR

1.1 Nya Koncept och Funktioner

1. Avancerad Objektmetadata (Slide 14 - NY)

NYTT SYSTEM FÖR OBJEKTHANTERING
- Omfattande metadatamodell för alla objekt i systemet
- Arvsstruktur för data genom klusterhierarkin (mamma-barn-relationer)
- Flexibel säkerhet och låsningsmekanismer för data
- Bildgalleri och dokumenthantering per objekt
- Omfattande metadata-typer för spårning och hantering

2. Avancerad Uppgifts- och Ordermetadata (Slide 15 - NY)

NYTT SYSTEM FÖR UPPGIFTSHANTERING
- Detaljerad metadata för varje uppgift/order
- Användarhantering med rättigheter (vem skapade, vem får ändra)
- Flera sätt att skapa/uppdatera data (manuellt, automatiskt, extern inmatning)
- Avancerad tidshantering med flera tidsfönster
- Beroendelogik mellan uppgifter
- GPS och What 3 words-integration för platsbestämning
- Omfattande statushantering (10+ statusnivåer)
- Flexibla faktureringsmodeller

3. Utökad Informationsvy (Slide 16 - NY)

NY VISNING
- Ny vy för “Uppgifter och information” kopplat till utförare/team

1.2 Ändringar i Befintliga Koncept

INGA ÄNDRINGAR - Alla befintliga koncept (slides 1-13) är oförändrade. Snöret-konceptet och status-exemplet är flyttade till slides 17-18 men har samma innehåll.

1.3 Borttagna Funktioner

INGET BORTTAGET - Ingen funktionalitet har tagits bort, endast utökad.

DEL 2: DETALJERAD BESKRIVNING AV VARJE ÄNDRING

2.1 Avancerad Objektmetadata

2.1.1 Vad är det?

Ett omfattande system för att hantera metadata kopplat till alla objekt i Modus 2.0. Objekt kan vara fysiska (t.ex. en matavfallsbehållare, en container, en återvinningsstation) eller organisatoriska (t.ex. en bostadsrättsförening, en byggnad, en gård). Systemet definierar vilken data som kan kopplas till objekt, hur den ärvs i hierarkin, och hur den skyddas.

2.1.2 Varför är det viktigt?

Affärsvärde:
- Möjliggör exakt spårning av varje enskilt objekt i Kinabs verksamhet
- Ger flexibilitet att definiera data på olika nivåer (BRF-nivå, byggnads-nivå, behållar-nivå)
- Säkerställer datakonsistens genom arvslogik
- Möjliggör historik och dokumentation per objekt (bildgalleri, protokoll)
- Ger kunden (t.ex. BRF) insyn i sina objekt via kundinloggning

Tekniskt värde:
- Tydlig datamodell för objekts metadata
- Flexibel hantering av både gemensam och unik data
- Säkerhetsmodell för att skydda känslig data
- Skalbar struktur som fungerar för olika objekttyper

2.1.3 Hur ska det fungera?

A. Unik Data för Objekt

Varje objekt har obligatoriska grunddata:
- **Namn**: T.ex. “Matavfallsbehållare Storgatan 1A”
- **Nummer**: Unikt ID/serienummer (t.ex. #23432143423)
- **Plats/Adress**: Exakt position där objektet finns
- **Mamma(or)**: Referens till föräldraobjekt i hierarkin
- **Kund(er)/Fakturamotagare**: Vilken kund som äger/betalar för objektet
- **Prislista/Rabattbrev/Avtal**: Vilka prisavtal som gäller
- **Objekttyp**: Organisatoriskt eller fysiskt
- **Bild/Icon**: Visuell representation

B. Data Kopplat till Objektet (Arvsmodell)

Data kan ha tre ursprung:
1. **Ärvd från mamma**: Data som kommer från föräldraobjekt (t.ex. prislista från BRF)
2. **Egen låst**: Data som är specifik för objektet och inte kan ändras av underliggande objekt
3. **Egen data som ärvs till barn**: Data som objektet definierar och som barnobjekt ärver

Arvslogik:

```
BRF Rosenlund (Prislista A, Kontakt: Johan)
  └─> Byggnad 1 (Ärver: Prislista A, Kontakt: Johan)
       └─> Matavfallsbehållare 23 (Ärver: Prislista A, Kontakt: Johan)
            └─> Kan ha egen data: Storlek: 240L (låst, ärvs inte vidare)
```

C. Datasäkerhet/Låsning

Tre nivåer av datasäkerhet:
1. **Fri**: Data kan ändras av vem som helst med rättigheter
2. **Låst**: Data kan inte ändras alls (skrivskyddad)
3. **Justerbar**: Data kan ändras men bara av specifika användare/roller

D. Typer av Löpande Data

Omfattande metadata som kan kopplas till objekt:
- **Objekttyp för artiklar att haka på**: Vilka typer av tjänster/artiklar som kan kopplas
- **Kopplade kontakter**: Vilka personer som ska kontaktas för objektet
- **Mängd/Antal/Storlek**: T.ex. “240 liter”, “2 st behållare”
- **Individ info/Märke/Nummer**: Tillverkarinfo, serienummer
- **Referensinformation**: Externa ID, kundnummer, etc.
- **Tillgänglighet**: När objektet är tillgängligt för service
- **Objektbild eller dokument**: Foton, ritningar, manualer
- **Tillgängliga önskade datum**: När service önskas
- **Protokoll/Status**: Aktuell status, historik
- **Bildgalleri med datum**: Fotodokumentation över tid
- **Kopplade uppgifter/Orderkoncept**: Vilka ordrar som är kopplade
- **Felanmälningar/Övrig info**: Rapporterade problem, anteckningar

2.1.4 Konkreta Exempel från Kinab

Exempel 1: Matavfallsbehållare

```
Objekt: Matavfallsbehållare Storgatan
Nummer: #23432143423
Typ: Fysiskt objekt
Plats: Storgatan 12A, 12345 Stockholm
GPS: 59.3293, 18.0686
What 3 words: ///exempel.adress.här

Mamma: BRF Rosenlund -> Byggnad A
Kund: BRF Rosenlund (Org.nr: 556123-4567)
Fakturamottagare: BRF Rosenlund
Kontakt: Johan Svensson (070-123 45 67)

Prislista: Standard BRF (ärvd från BRF Rosenlund)
Rabattbrev: Nej (ärvd från BRF Rosenlund)

Objekttyp för artiklar: "Matavfall", "Container", "Underhåll"
Mängd/Storlek: 240L (egen låst data)
Märke: Kärl & Co AB (egen data)
Serienummer: KC-240-8832 (egen data)
Tillgänglighet: Måndag-Fredag 07:00-16:00 (ärvd från BRF)

Bildgalleri:
  - 2025-01-04: Foto vid leverans (bildgalleri med datum)
  - 2025-03-15: Foto efter underhåll

Kopplade uppgifter:
  - Uppgift #4523: Tömning matavfall (veckovis)
  - Uppgift #4891: Underhåll lock (2025-04-20)

Felanmälningar:
  - 2025-03-10: Lock sitter hårt (Status: Åtgärdad)
```

Exempel 2: Klusterhierarki med Arvslogik

```
BRF Rosenlund (Organisatoriskt objekt)
  Prislista: Standard BRF (egen data som ärvs)
  Kontakt: Johan Svensson (egen data som ärvs)
  Rabatt: 5% på allt (egen data som ärvs)
  Tillgänglighet: Mån-Fre 07:00-16:00 (egen data som ärvs)

  └─> Byggnad A (Organisatoriskt objekt)
       Ärver: Prislista, Kontakt, Rabatt, Tillgänglighet
       Egen data: Adress: Storgatan 12A (ärvs till barn)

       └─> Port 1 (Organisatoriskt objekt)
            Ärver: Allt från Byggnad A
            Egen data: Portkod: 1234 (låst, ärvs inte)

            └─> Matavfallsbehållare #23432143423 (Fysiskt objekt)
                 Ärver: Prislista, Kontakt, Rabatt, Tillgänglighet, Adress
                 Egen låst data: Storlek: 240L, Märke: Kärl & Co
                 Egen data som ärvs: Ingen (detta är lägsta nivån)
```

2.2 Avancerad Uppgifts- och Ordermetadata

2.2.1 Vad är det?

Ett omfattande system för att hantera all metadata kopplat till uppgifter och ordrar i Modus 2.0. Detta definierar vilken information som kan kopplas till varje uppgift, hur uppgifter skapas, vem som kan ändra dem, hur de hänger ihop (beroendelogik), och hur de hanteras genom hela livscykeln från skapande till fakturering.

2.2.2 Varför är det viktigt?

Affärsvärde:
- Fullständig spårbarhet av varje uppgift (vem skapade, när, hur)
- Flexibel tidshantering med flera önskade tidsfönster (viktigt för kunder)
- Automatisk beroendelogik (t.ex. “föravisering måste ske innan tömning”)
- Exakt platsbestämning (GPS + What 3 words) för utförare
- Detaljerad statushantering för att följa uppgiftens progress
- Flexibla faktureringsmodeller (per uppgift, per order, per månad)
- Rättighetshantering (vem får ändra vad)

Tekniskt värde:
- Tydlig datamodell för uppgiftens livscykel
- Historik och loggning av alla ändringar
- Automatik för beroendeuppgifter (system skapar relaterade uppgifter automatiskt)
- Integration med externa system (felanmälan från kund)
- Flexibel och utökningsbar struktur

2.2.3 Hur ska det fungera?

A. Kopplat till Objekt

Varje uppgift är kopplad till:
- **Objekt-ID**: Vilket objekt uppgiften gäller
- **Artikelnummer**: Vilken artikel/tjänst (t.ex. “Tömning matavfall”)
- **Lås till objektnivå**: Kan uppgiften flyttas till annat objekt? (Ja/Nej)
- **Ärvd av högre klusternivå**: Skapades uppgiften från klusternivå? (Ja/Nej)

B. Användarhantering

Rättighetshantering för varje uppgift:
- **Skapat av**: Vilken användare/system som skapade uppgiften
- **Får ändras av**: Vilka användare/roller som får ändra uppgiften

C. Skapat Genom

Fem sätt att skapa/uppdatera uppgifter:
1. **Manuell inmatning**: Användare skapar manuellt i systemet
2. **Inläst data**: Importerad från Excel/CSV
3. **Extern inmatning**: T.ex. felanmälan från kund via webbportal
4. **Uppdaterat av utförare vid uppgift**: Utförare ändrar uppgiften på plats
5. **Automatik**: System skapar automatiskt (t.ex. beroendeuppgifter, strukturartiklar)

D. Tider (Avancerad Tidshantering)

Omfattande tidsrelaterad metadata:
- **Logg**: Fullständig historik av alla tidsändringar
- **Koppling**: Tidsstämpel när uppgiften kopplades (format: YYMMDDHHMM)

- **Slutfört**: När uppgiften slutfördes
- **Leverans - planerad**: Planerat leveransdatum
- **Önskad leverans**: Flera tidsfönster (t.ex. “Vecka 15 Måndag 08:00-12:00 ELLER Vecka 15 Onsdag 13:00-16:00”)
- **Leveranstid**: Angiven/beräknad tid från beställning till leverans
- **Utförandetid**: Hur lång tid själva arbetet tar
- **Ställ/res**: Tid för inställelse och resa

Viktigt: Flera tidsfönster möjliggör flexibilitet för kunden. Exempel:

```
Önskad leverans för Uppgift #4523:
  Alternativ 1: Vecka 15, Måndag, 08:00-12:00
  Alternativ 2: Vecka 15, Onsdag, 13:00-16:00
  Alternativ 3: Vecka 16, Tisdag, 07:00-10:00
```

E. Beroendelogik

Uppgifter kan vara beroende av varandra:
- **Kopplat till slutförd annan uppgift**: “Uppgift B kan bara utföras efter att Uppgift A är slutförd”
- **Skapas automatiskt av objektdata**: System skapar uppgiften baserat på objektets metadata
- **Ingår i strukturartikel**: Uppgiften är en del av en större struktur (t.ex. “Förplock” + “Tömning” + “Efterkontroll”)

Exempel beroendelogik:

```
Order: "Byte av matavfallsbehållare"
  └─> Strukturartikel med beroendeuppgifter:
       1. Föravisering (måste göras först)
       2. Hämtning av gammal behållare (beroende av #1)
       3. Leverans av ny behållare (beroende av #2)
       4. Efteravisering (beroende av #3)
```

F. Plats (Exakt Positionering)

Tre sätt att definiera plats:
1. **Gatuadress**: “Storgatan 12A, 12345 Stockholm”
2. **GPS-koordinater**: “59.3293, 18.0686”
3. **What 3 words**: “///exempel.adress.här” (exakt 3x3m ruta)

Varför What 3 words?
- Mycket exakt positionering (viktigt när det finns flera behållare på samma adress)
- Lättare för utförare att hitta exakt plats
- Fungerar även där gatuadresser är otydliga

G. Typ av Information

Information kan vara två typer:
1. **Information utan logik**:
- Text (fritext-anteckningar)
- Bilder (foton på plats)
- Filer (dokument, avtal)

1. **Information med logik koppling till artikel**:
   - Typ beskrivning för artikel att haka fast
   - Till/från artikel mängd/antal (material som behövs/används)

H. Order och Fakturering

Varje uppgift har faktureringsrelaterad metadata:
- **Beställning/order koppling**: Vilket ordernummer uppgiften tillhör
- **Faktureringsmodell**: Hur ska detta faktureras?
- Per uppgift
- Per order
- Månadsfaktura
- Enligt avtal
- **Referens**: För sökning och faktura
- Objektreferens
- Orderreferens
- Kundreferens

I. Status (Livscykelhantering)

Fyra huvudstatuskategorier:
1. **Aktivt**: Uppgiften är aktiv och synlig
2. **Synligt men ej kopplat**: Uppgiften finns men är inte kopplad till order/uppgift/faktura än
3. **Kopplat till order/uppgift/faktura**: Uppgiften är aktiv och kopplad
4. **Arkiverat – ej synligt**: Uppgiften är avslutad och arkiverad

J. Uppgiftstyp

Definierar vilken typ av utförare som kan utföra uppgiften:
- T.ex. “Sophämtning”, “Containerhantering”, “Underhåll”, “Felanmälan”
- Används för att matcha uppgift med rätt kompetens

K. Utförande Status (Detaljerad Workflow)

10 statusnivåer genom uppgiftens livscykel:
1. **Ej planerat**: Uppgiften är skapad men inte inplanerad
2. **Planerat grovt (Vecka)**: Inplanerad i en specifik vecka
3. **Planerat fint (dag och tid i snöre)**: Inplanerad på exakt dag och tid
4. **På väg**: Utförare är på väg till platsen
5. **På plats**: Utförare har anlänt
6. **Utfört**: Arbetet är utfört
7. **Kontroll**: Uppgiften är under kontroll/granskning
8. **Fakturerat**: Uppgiften är fakturerad

Workflow:

```
Ej planerat → Planerat grovt → Planerat fint → På väg → På plats → Utfört → Kontroll →
Fakturerat
```

L. Tilldelning

Uppgiften kan tilldelas på olika nivåer:
- **Distrikt**: T.ex. “Stockholm Söder”
- **Utförare/team**: Specifik person eller team
- **KS (utr)**: Koordinator/Systemansvarig
- **Proj (pers)**: Projektansvarig person

2.2.4 Konkreta Exempel från Kinab

Exempel 1: Tömning av Matavfallsbehållare

```
UPPGIFT #4523: Tömning matavfall

--- KOPPLAT TILL OBJEKT ---
Objekt-ID: #23432143423 (Matavfallsbehållare Storgatan)
Artikelnummer: ART-100 (Tömning matavfall 240L)
Lås till objektnivå: Ja
Ärvd av högre klusternivå: Nej

--- ANVÄNDARHANTERING ---
Skapat av: System (Automatik från orderkoncept)
Får ändras av: Planerare, Koordinator
Skapat genom: Automatik (Beroendeuppgift/vara)

--- TIDER ---
Logg:
  2025-01-04 10:00 - Uppgift skapad
  2025-01-05 14:30 - Planerat grovt till Vecka 15
  2025-01-08 09:15 - Planerat fint till 2025-04-07 08:00
Koppling: 2501041000
Slutfört: (ännu ej slutförd)
Leverans - planerad: 2025-04-07
Önskad leverans:
  Alt 1: Vecka 15, Måndag, 08:00-12:00
  Alt 2: Vecka 15, Onsdag, 13:00-16:00
Leveranstid: 2 dagar (från beställning)
Utförandetid: 0.5h (30 min)
Ställ/res: 0.2h (12 min resa)

--- BEROENDELOGIK ---
Kopplat till slutförd annan uppgift: Nej
Skapas automatiskt av objektdata: Ja (veckovis återkommande)
Ingår i strukturartikel: Ja
  └─> Del av: "Matavfallshantering"
        1. Föravisering (denna uppgift är beroende)
        2. Tömning (DENNA UPPGIFT)
        3. Efterkontroll (beroende av denna)

--- PLATS ---
Gatuadress: Storgatan 12A, 12345 Stockholm
GPS: 59.3293, 18.0686
What 3 words: ///exempel.adress.här

--- TYP AV INFORMATION ---
Information utan logik:
  Text: "OBS! Smalt infart, använd mindre fordon"
  Bilder: (1) Foto på plats från 2025-01-04
Information med logik:
  Typ beskrivning: "Matavfall"
  Mängd: 240L (estimerad full)

--- ORDER OCH FAKTURERING ---
Beställning/order koppling: ORDER-2025-0234
Faktureringsmodell: Månadsfaktura
Referens:
  Objektreferens: #23432143423
  Orderreferens: ORDER-2025-0234
  Kundreferens: BRF Rosenlund

--- STATUS ---
Status: Kopplat till order/uppgift/faktura

--- UPPGIFTSTYP ---
```

```
Uppgiftstyp: Sophämtning

--- UTFÖRANDE STATUS ---
Utförande status: Planerat fint (dag och tid i snöre)

--- TILLDELNING ---
Distrikt: Stockholm Söder
Utförare/team: Team B (Johan + Erik)
KS (utr): Anna Andersson
Proj (pers): -
```

Exempel 2: Extern Felanmälan från Kund

```
UPPGIFT #4891: Underhåll - Lock sitter hårt

--- KOPPLAT TILL OBJEKT ---
Objekt-ID: #23432143423 (Matavfallsbehållare Storgatan)
Artikelnummer: ART-500 (Underhåll behållare)
Lås till objektnivå: Ja
Ärvd av högre klusternivå: Nej

--- ANVÄNDARHANTERING ---
Skapat av: Johan Svensson (BRF Rosenlund kontakt) via kundportal
Får ändras av: Planerare, Koordinator, Servicetekniker
Skapat genom: Extern inmatning (felanmälan)

--- TIDER ---
Logg:
  2025-03-10 14:23 - Felanmälan inkom via kundportal
  2025-03-10 15:00 - Uppgift skapad automatiskt
  2025-03-11 09:00 - Tilldelad till Servicetekniker Lisa
  2025-03-11 10:30 - Planerat till 2025-03-12 13:00
Koppling: 2503101500
Slutfört: 2025-03-12 14:30
Leverans - planerad: 2025-03-12
Önskad leverans: "Så snart som möjligt"
Leveranstid: 2 dagar (från felanmälan)
Utförandetid: 1.0h
Ställ/res: 0.3h

--- BEROENDELOGIK ---
Kopplat till slutförd annan uppgift: Nej
Skapas automatiskt av objektdata: Ja (från felanmälan)
Ingår i strukturartikel: Nej

--- PLATS ---
Gatuadress: Storgatan 12A, 12345 Stockholm
GPS: 59.3293, 18.0686
What 3 words: ///exempel.adress.här

--- TYP AV INFORMATION ---
Information utan logik:
  Text: "Lock sitter hårt och går inte att öppna. Kund rapporterar att det varit så i
en vecka."
  Bilder: (2) Foto från kund via kundportal
Information med logik:
  Typ beskrivning: "Underhåll - Lock"
  Material: Lock-reservdel (om behövs)

--- ORDER OCH FAKTURERING ---
Beställning/order koppling: ORDER-2025-0589 (Skapad automatiskt från felanmälan)
Faktureringsmodell: Per uppgift
Referens:
  Objektreferens: #23432143423
  Orderreferens: ORDER-2025-0589
  Kundreferens: BRF Rosenlund
  Felanmälningsreferens: FELANM-2025-0123

--- STATUS ---
Status: Kopplat till order/uppgift/faktura

--- UPPGIFTSTYP ---
Uppgiftstyp: Underhåll

--- UTFÖRANDE STATUS ---
```

```
Utförande status: Fakturerat

--- TILLDELNING ---
Distrikt: Stockholm Söder
Utförare/team: Lisa Nilsson (Servicetekniker)
KS (utr): Anna Andersson
Proj (pers): -
```

Exempel 3: Strukturartikel med Beroendeuppgifter

```
ORDER: Byte av matavfallsbehållare #23432143423
Ordernummer: ORDER-2025-0678

STRUKTURARTIKEL: "Byte av behållare 240L"
  └─> Automatiskt skapade beroendeuppgifter:

1. UPPGIFT #5001: Föravisering
   Utförande status: Utfört
   Slutfört: 2025-04-01 10:00
   Skapat genom: Automatik (Beroendeuppgift/vara)
   Beroendelogik: Ingår i strukturartikel, måste göras först
   Typ: Telefon/SMS till kund

2. UPPGIFT #5002: Hämtning gammal behållare
   Utförande status: Utfört
   Slutfört: 2025-04-02 09:30
   Skapat genom: Automatik (Beroendeuppgift/vara)
   Beroendelogik: Kopplat till slutförd uppgift #5001
   Utförandetid: 0.7h

3. UPPGIFT #5003: Leverans ny behållare
   Utförande status: Utfört
   Slutfört: 2025-04-02 09:50
   Skapat genom: Automatik (Beroendeuppgift/vara)
   Beroendelogik: Kopplat till slutförd uppgift #5002
   Utförandetid: 0.5h

4. UPPGIFT #5004: Efteravisering
   Utförande status: Utfört
   Slutfört: 2025-04-02 14:00
   Skapat genom: Automatik (Beroendeuppgift/vara)
   Beroendelogik: Kopplat till slutförd uppgift #5003
   Typ: SMS till kund med bekräftelse
```

2.3 Utökad Informationsvy (Slide 16)

2.3.1 Vad är det?

En ny vy i systemet som samlar och presenterar “Uppgifter och information” specifikt kopplat till utförare/team.

2.3.2 Varför är det viktigt?

- Ger utförare en samlad bild av sina uppgifter
- Fokuserad vy på relevant information för utföraren
- Underlättar för utföraren att se vad som ska göras

2.3.3 Hur ska det fungera?

En dedikerad vy där utförare/team ser:
- Sina tilldelade uppgifter
- All relevant information per uppgift
- Koppling till objekt och metadata
- Status och prioritet

2.3.4 Konkreta Exempel från Kinab

```
VY: Uppgifter och information - Team B (Johan + Erik)

Vecka 15 - Måndag 2025-04-07
═══════════════════════════════════════

08:00-08:12  RESA: Från Kinab → Storgatan 12A
             Avstånd: 12 km
             What 3 words: ///exempel.adress.här

08:12-08:42  UPPGIFT #4523: Tömning matavfall
             Objekt: Matavfallsbehållare #23432143423
             Plats: Storgatan 12A (GPS: 59.3293, 18.0686)
             Kund: BRF Rosenlund (Johan Svensson, 070-123 45 67)
             Status: Planerat fint
             OBS! Smalt infart, använd mindre fordon

08:42-09:10  RESA: Storgatan 12A → Rosenlundsvägen 45
             Avstånd: 18 km

09:10-10:00  UPPGIFT #4567: Byte av container
             Objekt: Container #98765
             Plats: Rosenlundsvägen 45
             ...
```

DEL 3: IMPLEMENTATIONSINSTRUKTIONER FÖR REPLIT 3.0

3.1 Avancerad Objektmetadata

3.1.1 Vad behöver byggas?

Databas Schema

Tabell: `object_metadata`

```sql
CREATE TABLE object_metadata (
  id SERIAL PRIMARY KEY,
  object_id INTEGER REFERENCES objects(id),
  metadata_key VARCHAR(100) NOT NULL,
  metadata_value TEXT,
  data_source VARCHAR(50), -- 'inherited', 'own_locked', 'own_inheritable'
  security_level VARCHAR(20), -- 'free', 'locked', 'adjustable'
  created_by INTEGER REFERENCES users(id),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_object_metadata_object ON object_metadata(object_id);
CREATE INDEX idx_object_metadata_key ON object_metadata(metadata_key);
```

Uppdatera: `objects` tabell

```sql
ALTER TABLE objects ADD COLUMN IF NOT EXISTS name VARCHAR(255);
ALTER TABLE objects ADD COLUMN IF NOT EXISTS number VARCHAR(100) UNIQUE;
ALTER TABLE objects ADD COLUMN IF NOT EXISTS address TEXT;
ALTER TABLE objects ADD COLUMN IF NOT EXISTS parent_id INTEGER REFERENCES objects(id);
ALTER TABLE objects ADD COLUMN IF NOT EXISTS customer_id INTEGER REFERENCES custom-
ers(id);
ALTER TABLE objects ADD COLUMN IF NOT EXISTS pricelist_id INTEGER REFERENCES pricel-
ists(id);
ALTER TABLE objects ADD COLUMN IF NOT EXISTS object_type VARCHAR(50); -- 'organiza-
tional' or 'physical'
ALTER TABLE objects ADD COLUMN IF NOT EXISTS image_url VARCHAR(500);
ALTER TABLE objects ADD COLUMN IF NOT EXISTS icon_url VARCHAR(500);
```

Ny tabell: `object_images`

```sql
CREATE TABLE object_images (
  id SERIAL PRIMARY KEY,
  object_id INTEGER REFERENCES objects(id),
  image_url VARCHAR(500),
  image_date DATE,
  description TEXT,
  uploaded_by INTEGER REFERENCES users(id),
  uploaded_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_object_images_object ON object_images(object_id);
CREATE INDEX idx_object_images_date ON object_images(image_date);
```

Ny tabell: `object_contacts`

```sql
CREATE TABLE object_contacts (
  id SERIAL PRIMARY KEY,
  object_id INTEGER REFERENCES objects(id),
  contact_id INTEGER REFERENCES contacts(id),
  contact_type VARCHAR(50), -- 'primary', 'secondary', 'billing', etc.
  inherited_from INTEGER REFERENCES objects(id), -- NULL if not inherited
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_object_contacts_object ON object_contacts(object_id);
```

3.1.2 Vilka datamodeller behöver skapas/uppdateras?

Datamodell: `ObjectMetadata`

```python
class ObjectMetadata:
    def __init__(self, object_id, key, value, source, security_level):
        self.object_id = object_id
        self.key = key
        self.value = value
        self.source = source  # 'inherited', 'own_locked', 'own_inheritable'
        self.security_level = security_level  # 'free', 'locked', 'adjustable'
        self.created_by = None
        self.created_at = None
        self.updated_at = None
```

Uppdatera: Datamodell `Object`

```
class Object:
    def __init__(self, id, name, number, address, parent_id, customer_id,
                 pricelist_id, object_type, image_url, icon_url):
        self.id = id
        self.name = name
        self.number = number  # Unikt nummer
        self.address = address
        self.parent_id = parent_id  # Mamma
        self.customer_id = customer_id
        self.pricelist_id = pricelist_id
        self.object_type = object_type  # 'organizational' or 'physical'
        self.image_url = image_url
        self.icon_url = icon_url

        # Relations
        self.metadata = []  # List of ObjectMetadata
        self.images = []  # List of ObjectImage
        self.contacts = []  # List of ObjectContact

    def get_metadata(self, key, include_inherited=True):
        """Hämta metadata med arvslogik"""
        # Logic to fetch metadata, considering inheritance
        pass

    def set_metadata(self, key, value, source='own_locked', security_level='free'):
        """Sätt metadata för objekt"""
        pass

    def get_inherited_metadata(self):
        """Hämta all ärvd metadata från förälder"""
        pass

    def get_inheritable_metadata(self):
        """Hämta metadata som kan ärvas av barn"""
        pass
```

Ny datamodell: `ObjectImage`

```
class ObjectImage:
    def __init__(self, id, object_id, image_url, image_date, description,
uploaded_by):
        self.id = id
        self.object_id = object_id
        self.image_url = image_url
        self.image_date = image_date
        self.description = description
        self.uploaded_by = uploaded_by
        self.uploaded_at = None
```

3.1.3 Vilka funktioner behöver implementeras?

Funktion: Arvslogik för Metadata

```
def get_metadata_with_inheritance(object_id, metadata_key):
    """
    Hämta metadata för objekt med arvslogik.

    Logik:
    1. Kolla om objektet har egen metadata för denna key
    2. Om egen metadata finns OCH är 'own_locked' eller 'own_inheritable': returnera
den
    3. Om ingen egen metadata: gå upp till förälder (parent_id) och rekursivt hämta
    4. Om förälder har metadata med source='own_inheritable': returnera den
    5. Fortsätt upp i hierarkin tills metadata hittas eller root nås
    """
    obj = get_object(object_id)

    # Kolla egen metadata först
    own_metadata = get_own_metadata(object_id, metadata_key)
    if own_metadata and own_metadata.source in ['own_locked', 'own_inheritable']:
        return own_metadata

    # Om ingen egen, kolla förälder
    if obj.parent_id:
        parent_metadata = get_metadata_with_inheritance(obj.parent_id, metadata_key)
        if parent_metadata and parent_metadata.source == 'own_inheritable':
            # Returnera som inherited
            return ObjectMetadata(
                object_id=object_id,
                key=metadata_key,
                value=parent_metadata.value,
                source='inherited',
                security_level=parent_metadata.security_level
            )

    return None
```

Funktion: Sätt Metadata med Säkerhetskontroll

```
def set_metadata(object_id, metadata_key, value, user_id,
                 source='own_locked', security_level='free'):
    """
    Sätt metadata för objekt med säkerhetskontroll.

    Säkerhetskontroller:
    1. Om metadata redan finns och är 'locked': Kontrollera om användaren har rät-
tighet
    2. Om 'adjustable': Kontrollera om användaren är i tillåten grupp
    3. Om 'free': Tillåt ändring
    """
    existing = get_own_metadata(object_id, metadata_key)

    if existing:
        # Säkerhetskontroll
        if existing.security_level == 'locked':
            if not user_has_admin_rights(user_id):
                raise PermissionError("Metadata är låst och kan inte ändras")
        elif existing.security_level == 'adjustable':
            if not user_can_adjust_metadata(user_id, object_id):
                raise PermissionError("Du har inte rättighet att ändra denna
metadata")

        # Uppdatera befintlig
        update_metadata(existing.id, value, user_id)
    else:
        # Skapa ny
        create_metadata(object_id, metadata_key, value, source, security_level,
user_id)
```

Funktion: Hämta Bildgalleri för Objekt

```
def get_object_image_gallery(object_id, start_date=None, end_date=None):
    """
    Hämta bildgalleri för objekt, sorterat efter datum.

    Parametrar:
    - object_id: Objektets ID
    - start_date: Från datum (optional)
    - end_date: Till datum (optional)

    Returnerar: Lista av ObjectImage sorterat efter image_date DESC
    """
    query = "SELECT * FROM object_images WHERE object_id = %s"
    params = [object_id]

    if start_date:
        query += " AND image_date >= %s"
        params.append(start_date)

    if end_date:
        query += " AND image_date <= %s"
        params.append(end_date)

    query += " ORDER BY image_date DESC"

    return execute_query(query, params)
```

Funktion: Kopiera Kluster med Arvslogik

```
def copy_cluster_with_inheritance(source_cluster_id, target_parent_id=None):
    """
    Kopiera ett helt kluster inklusive arvslogik.

    Logik:
    1. Kopiera root-objektet
    2. För varje barn-objekt:
       a. Kopiera objektet
       b. Kopiera egen metadata (own_locked, own_inheritable)
       c. Skippa inherited metadata (kommer automatiskt från förälder)
    3. Rekursivt kopiera alla barn
    """
    # Implementation här
    pass
```

3.1.4 Hur ska det integreras med befintligt system?

Integration A: Objekt-skapande

När ett nytt objekt skapas:
1. Spara grunddata i `objects` tabell
2. Om objektet har en förälder (`parent_id`): Hämta föräldrens `own_inheritable` metadata och skapa motsvarande `inherited` metadata för barnet
3. Skapa eventuell egen metadata

Integration B: Klusterhierarki

När klusterhierarkin ändras (objekt flyttas):
1. Ta bort all `inherited` metadata för objektet
2. Hämta ny förälders `own_inheritable` metadata
3. Skapa ny `inherited` metadata baserat på ny förälder
4. Rekursivt uppdatera alla barn till objektet

Integration C: Orderkoncept och Uppgifter

När uppgifter skapas från orderkoncept:
1. Hämta metadata från objekt (inklusive inherited)
2. Använd metadata för att populera uppgiftens fält (plats, tid, kontakt, etc.)

Integration D: Kundinloggning

När kund loggar in:
1. Visa objektens metadata (beroende på rättigheter)
2. Visa bildgalleri för objekten
3. Tillåt kunden att skapa felanmälningar som skapar ny metadata

3.1.5 Prioritet

KRITISK - Detta är fundamental funktionalitet för hela systemet

---

3.2 Avancerad Uppgifts- och Ordermetadata

3.2.1 Vad behöver byggas?

Databas Schema

Uppdatera: `tasks` tabell (uppgifter)

```
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS object_id INTEGER REFERENCES objects(id);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS article_number VARCHAR(100);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS locked_to_object BOOLEAN DEFAULT FALSE;
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS inherited_from_cluster BOOLEAN DEFAULT FALS
E;
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS created_by INTEGER REFERENCES users(id);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS editable_by_role VARCHAR(100); -- Comma-
separated roles
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS creation_method VARCHAR(50);
-- 'manual', 'imported', 'external', 'performer_update', 'automatic'
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS time_log JSONB; -- För tidshistorik
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS connection_timestamp VARCHAR(20); -- YYMM-
DDHHMM
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS completed_at TIMESTAMP;
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS planned_delivery_date DATE;
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS delivery_time_days INTEGER;
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS execution_time_hours DECIMAL(5,2);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS travel_time_hours DECIMAL(5,2);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS address TEXT;
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS gps_lat DECIMAL(10,8);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS gps_lng DECIMAL(11,8);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS what3words VARCHAR(100);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS order_id INTEGER REFERENCES orders(id);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS billing_model VARCHAR(50);
-- 'per_task', 'per_order', 'monthly', 'contract'
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS reference_object VARCHAR(200);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS reference_order VARCHAR(200);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS reference_customer VARCHAR(200);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS status VARCHAR(50);
-- 'active', 'visible_unconnected', 'connected', 'archived'
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS task_type VARCHAR(100);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS execution_status VARCHAR(50);
-- 'not_planned', 'planned_rough', 'planned_fine', 'on_way', 'on_site',
-- 'completed', 'inspected', 'invoiced'
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS assigned_district VARCHAR(100);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS assigned_team VARCHAR(100);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS assigned_coordinator INTEGER REFERENCES use
rs(id);
ALTER TABLE tasks ADD COLUMN IF NOT EXISTS assigned_project_manager INTEGER
REFERENCES users(id);
```

Ny tabell: `task_desired_timewindows`

```sql
CREATE TABLE task_desired_timewindows (
  id SERIAL PRIMARY KEY,
  task_id INTEGER REFERENCES tasks(id),
  week_number INTEGER,
  day_of_week VARCHAR(20), -- 'monday', 'tuesday', etc.
  start_time TIME,
  end_time TIME,
  priority INTEGER, -- 1 = highest priority
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_task_timewindows_task ON task_desired_timewindows(task_id);
CREATE INDEX idx_task_timewindows_week ON task_desired_timewindows(week_number);
```

Ny tabell: `task_dependencies`

```sql
CREATE TABLE task_dependencies (
  id SERIAL PRIMARY KEY,
  task_id INTEGER REFERENCES tasks(id), -- Den uppgift som är beroende
  depends_on_task_id INTEGER REFERENCES tasks(id), -- Den uppgift som måste slutföras
först
  dependency_type VARCHAR(50), -- 'sequential', 'structural', 'automatic'
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_task_dependencies_task ON task_dependencies(task_id);
CREATE INDEX idx_task_dependencies_depends ON task_dependencies(depends_on_task_id);
```

Ny tabell: `task_information`

```sql
CREATE TABLE task_information (
  id SERIAL PRIMARY KEY,
  task_id INTEGER REFERENCES tasks(id),
  info_type VARCHAR(50), -- 'text', 'image', 'file', 'article_link'
  info_value TEXT,
  has_logic BOOLEAN DEFAULT FALSE,
  created_by INTEGER REFERENCES users(id),
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_task_info_task ON task_information(task_id);
CREATE INDEX idx_task_info_type ON task_information(info_type);
```

3.2.2 Vilka datamodeller behöver skapas/uppdateras?

Uppdatera: Datamodell `Task`

```
class Task:
    def __init__(self):
        # Befintliga fält (behålls)
        self.id = None
        self.name = None
        self.description = None

        # NYA fält - Kopplat till objekt
        self.object_id = None
        self.article_number = None
        self.locked_to_object = False
        self.inherited_from_cluster = False

        # NYA fält - Användarhantering
        self.created_by = None
        self.editable_by_role = []  # Lista av roller som får ändra
        self.creation_method = None  # 'manual', 'imported', 'external', 'per-
former_update', 'automatic'

        # NYA fält - Tider
        self.time_log = []  # Historik av alla tidsändringar
        self.connection_timestamp = None  # YYMMDDHHMM
        self.completed_at = None
        self.planned_delivery_date = None
        self.desired_delivery_timewindows = []  # Lista av TimeWindow
        self.delivery_time_days = None
        self.execution_time_hours = None
        self.travel_time_hours = None

        # NYA fält - Beroendelogik
        self.dependencies = []  # Lista av TaskDependency
        self.is_automatic = False
        self.is_structural = False

        # NYA fält - Plats
        self.address = None
        self.gps_lat = None
        self.gps_lng = None
        self.what3words = None

        # NYA fält - Information
        self.information = []  # Lista av TaskInformation

        # NYA fält - Order och fakturering
        self.order_id = None
        self.billing_model = None  # 'per_task', 'per_order', 'monthly', 'contract'
        self.reference_object = None
        self.reference_order = None
        self.reference_customer = None

        # NYA fält - Status
        self.status = None  # 'active', 'visible_unconnected', 'connected', 'archived'
        self.task_type = None
        self.execution_status = None  # 'not_planned', 'planned_rough', ..., 'in-
voiced'

        # NYA fält - Tilldelning
        self.assigned_district = None
        self.assigned_team = None
        self.assigned_coordinator = None
        self.assigned_project_manager = None
```

Ny datamodell: `DesiredTimeWindow`

```
class DesiredTimeWindow:
    def __init__(self, week_number, day_of_week, start_time, end_time, priority):
        self.week_number = week_number
        self.day_of_week = day_of_week
        self.start_time = start_time
        self.end_time = end_time
        self.priority = priority  # 1 = highest

    def __str__(self):
        return f"Vecka {self.week_number}, {self.day_of_week}, {self.start_time}-
{self.end_time}"
```

Ny datamodell: `TaskDependency`

```
class TaskDependency:
    def __init__(self, task_id, depends_on_task_id, dependency_type):
        self.task_id = task_id
        self.depends_on_task_id = depends_on_task_id
        self.dependency_type = dependency_type  # 'sequential', 'structural', 'auto-
matic'
```

Ny datamodell: `TaskInformation`

```
class TaskInformation:
    def __init__(self, task_id, info_type, info_value, has_logic, created_by):
        self.task_id = task_id
        self.info_type = info_type  # 'text', 'image', 'file', 'article_link'
        self.info_value = info_value
        self.has_logic = has_logic  # True för artikel-kopplingar, False för fritext
        self.created_by = created_by
        self.created_at = None
```

3.2.3 Vilka funktioner behöver implementeras?

Funktion: Skapa Uppgift med Full Metadata

```
def create_task_with_metadata(task_data, user_id, creation_method='manual'):
    """
    Skapa uppgift med all metadata.

    Parametrar:
    - task_data: Dictionary med all uppgiftsdata
    - user_id: Användarens ID
    - creation_method: 'manual', 'imported', 'external', 'performer_update', 'automat-
ic'

    Returnerar: Task objekt
    """
    # Validera rättigheter
    if not user_can_create_task(user_id, task_data['object_id']):
        raise PermissionError("Du har inte rättighet att skapa uppgift för detta ob-
jekt")

    # Hämta objektets metadata för att populera uppgiften
    obj = get_object(task_data['object_id'])
    obj_metadata = get_all_metadata_with_inheritance(obj.id)

    # Skapa uppgift
    task = Task()
    task.object_id = task_data['object_id']
    task.article_number = task_data.get('article_number')
    task.created_by = user_id
    task.creation_method = creation_method
    task.connection_timestamp = datetime.now().strftime('%y%m%d%H%M')

    # Populera från objektmetadata
    task.address = obj_metadata.get('address') or obj.address
    task.gps_lat = obj_metadata.get('gps_lat')
    task.gps_lng = obj_metadata.get('gps_lng')
    task.what3words = obj_metadata.get('what3words')

    # Spara uppgift
    task_id = save_task(task)

    # Skapa önskade tidsfönster
    if 'desired_timewindows' in task_data:
        for tw in task_data['desired_timewindows']:
            create_desired_timewindow(task_id, tw)

    # Logga skapande
    log_task_time_change(task_id, 'created', user_id)

    return get_task(task_id)
```

Funktion: Hantera Flera Önskade Tidsfönster

```
def add_desired_timewindow(task_id, week_number, day_of_week,
                           start_time, end_time, priority=1):
    """
    Lägg till ett önskat tidsfönster för uppgift.

    Exempel:
    add_desired_timewindow(4523, 15, 'monday', '08:00', '12:00', 1)
    add_desired_timewindow(4523, 15, 'wednesday', '13:00', '16:00', 2)
    add_desired_timewindow(4523, 16, 'tuesday', '07:00', '10:00', 3)
    """
    tw = DesiredTimeWindow(week_number, day_of_week, start_time, end_time, priority)

    query = """
        INSERT INTO task_desired_timewindows
        (task_id, week_number, day_of_week, start_time, end_time, priority)
        VALUES (%s, %s, %s, %s, %s, %s)
    """
    execute_query(query, [task_id, week_number, day_of_week, start_time, end_time,
priority])

def get_desired_timewindows(task_id):
    """
    Hämta alla önskade tidsfönster för uppgift, sorterat efter prioritet.
    """
    query = """
        SELECT * FROM task_desired_timewindows
        WHERE task_id = %s
        ORDER BY priority ASC
    """
    return execute_query(query, [task_id])
```

Funktion: Beroendelogik mellan Uppgifter

```python
def create_task_dependency(task_id, depends_on_task_id, dependency_type='sequential'):
    """
    Skapa beroende mellan två uppgifter.

    Parametrar:
    - task_id: Uppgiften som är beroende
    - depends_on_task_id: Uppgiften som måste slutföras först
    - dependency_type: 'sequential', 'structural', 'automatic'
    """
    # Kontrollera att beroende inte skapar cirkulär referens
    if creates_circular_dependency(task_id, depends_on_task_id):
        raise ValueError("Cirkulärt beroende är inte tillåtet")

    dep = TaskDependency(task_id, depends_on_task_id, dependency_type)

    query = """
        INSERT INTO task_dependencies (task_id, depends_on_task_id, dependency_type)
        VALUES (%s, %s, %s)
    """
    execute_query(query, [task_id, depends_on_task_id, dependency_type])

def can_task_be_executed(task_id):
    """
    Kontrollera om uppgift kan utföras (alla beroenden uppfyllda).

    Returnerar: (bool, list_of_blocking_tasks)
    """
    query = """
        SELECT td.depends_on_task_id, t.execution_status
        FROM task_dependencies td
        JOIN tasks t ON td.depends_on_task_id = t.id
        WHERE td.task_id = %s
    """
    dependencies = execute_query(query, [task_id])

    blocking_tasks = []
    for dep in dependencies:
        if dep['execution_status'] not in ['completed', 'inspected', 'invoiced']:
            blocking_tasks.append(dep['depends_on_task_id'])

    can_execute = len(blocking_tasks) == 0
    return can_execute, blocking_tasks

def get_dependent_tasks(task_id):
    """
    Hämta alla uppgifter som är beroende av denna uppgift.
    """
    query = """
        SELECT task_id FROM task_dependencies
        WHERE depends_on_task_id = %s
    """
    return execute_query(query, [task_id])
```

Funktion: Strukturartiklar med Automatiska Beroendeuppgifter

```
def create_structural_article_tasks(order_id, object_id, article_structure_id,
user_id):
    """
    Skapa alla uppgifter för en strukturartikel med automatiska beroenden.

    Exempel: "Byte av behållare" skapar:
    1. Föravisering
    2. Hämtning gammal behållare (beroende av #1)
    3. Leverans ny behållare (beroende av #2)
    4. Efteravisering (beroende av #3)
    """
    # Hämta strukturartikeln och dess delartiklar
    structure = get_article_structure(article_structure_id)

    created_tasks = []
    previous_task_id = None

    for step in structure.steps:
        # Skapa uppgift för varje steg
        task_data = {
            'object_id': object_id,
            'article_number': step.article_number,
            'order_id': order_id,
            'name': step.name,
            'task_type': step.task_type
        }

        task = create_task_with_metadata(task_data, user_id, creation_method='automat-
ic')

        # Skapa beroende till föregående uppgift
        if previous_task_id:
            create_task_dependency(task.id, previous_task_id, dependency_type='struc-
tural')

        created_tasks.append(task)
        previous_task_id = task.id

    return created_tasks
```

Funktion: GPS och What3Words Integration

```
def set_task_location(task_id, address=None, gps_lat=None, gps_lng=None, what3-
words=None):
    """
    Sätt plats för uppgift med olika format.

    Parametrar:
    - address: Gatuadress
    - gps_lat, gps_lng: GPS-koordinater
    - what3words: What3Words adress (format: ///ord.ord.ord)
    """
    update_fields = []
    params = []

    if address:
        update_fields.append("address = %s")
        params.append(address)

    if gps_lat and gps_lng:
        update_fields.append("gps_lat = %s, gps_lng = %s")
        params.extend([gps_lat, gps_lng])

    if what3words:
        # Validera What3Words format
        if not what3words.startswith('///') or what3words.count('.') != 2:
            raise ValueError("What3Words måste ha format: ///ord.ord.ord")

        update_fields.append("what3words = %s")
        params.append(what3words)

    if update_fields:
        query = f"UPDATE tasks SET {', '.join(update_fields)} WHERE id = %s"
        params.append(task_id)
        execute_query(query, params)

def geocode_address_to_gps(address):
    """
    Konvertera gatuadress till GPS-koordinater (via geocoding API).
    """
    # Integration med geocoding service (t.ex. Google Maps API)
    pass

def what3words_to_gps(what3words):
    """
    Konvertera What3Words till GPS-koordinater (via What3Words API).
    """
    # Integration med What3Words API
    pass
```

Funktion: Avancerad Statushantering

```
def update_task_execution_status(task_id, new_status, user_id):
    """
    Uppdatera uppgiftens utförande-status med validering.

    Tillåtna övergångar:
    not_planned -> planned_rough
    planned_rough -> planned_fine
    planned_fine -> on_way
    on_way -> on_site
    on_site -> completed
    completed -> inspected
    inspected -> invoiced
    """
    allowed_transitions = {
        'not_planned': ['planned_rough'],
        'planned_rough': ['planned_fine', 'not_planned'],
        'planned_fine': ['on_way', 'planned_rough'],
        'on_way': ['on_site', 'planned_fine'],
        'on_site': ['completed', 'on_way'],
        'completed': ['inspected'],
        'inspected': ['invoiced', 'completed'],
        'invoiced': []  # Terminal state
    }

    task = get_task(task_id)
    current_status = task.execution_status

    if new_status not in allowed_transitions.get(current_status, []):
        raise ValueError(f"Ogiltig statusövergång: {current_status} -> {new_status}")

    # Kontrollera beroenden innan status ändras till on_way eller on_site
    if new_status in ['on_way', 'on_site']:
        can_execute, blocking = can_task_be_executed(task_id)
        if not can_execute:
            raise ValueError(f"Kan inte utföra uppgift, beroende uppgifter ej slut-
förda: {blocking}")

    # Uppdatera status
    query = "UPDATE tasks SET execution_status = %s WHERE id = %s"
    execute_query(query, [new_status, task_id])

    # Logga statusändring
    log_task_time_change(task_id, f'status_change_{new_status}', user_id)

    # Om uppgift slutförd: kontrollera om beroende uppgifter kan påbörjas
    if new_status == 'completed':
        notify_dependent_tasks(task_id)
```

Funktion: Extern Inmatning (Felanmälan från Kund)

```python
def create_task_from_external_report(report_data, customer_id):
    """
    Skapa uppgift från extern felanmälan (t.ex. från kundportal).

    Parametrar:
    - report_data: Dictionary med felanmälningsdata
      {
        'object_id': 23432143423,
        'description': 'Lock sitter hårt',
        'images': ['url1', 'url2'],
        'priority': 'high',
        'contact_name': 'Johan Svensson',
        'contact_phone': '070-123 45 67'
      }
    - customer_id: Kundens ID

    Workflow:
    1. Validera att kunden har rättighet till objektet
    2. Skapa order automatiskt
    3. Skapa uppgift från felanmälan
    4. Lägg till information (text + bilder)
    5. Sätt status till 'active'
    6. Skicka bekräftelse till kund
    """
    # Validera rättigheter
    if not customer_owns_object(customer_id, report_data['object_id']):
        raise PermissionError("Kunden har inte rättighet till detta objekt")

    # Skapa order automatiskt
    order = create_order({
        'customer_id': customer_id,
        'order_type': 'external_report',
        'created_by': customer_id,
        'creation_method': 'external'
    })

    # Skapa uppgift
    task_data = {
        'object_id': report_data['object_id'],
        'article_number': determine_article_from_report(report_data['description']),
        'order_id': order.id,
        'name': f"Felanmälan: {report_data['description'][:50]}",
        'task_type': 'maintenance',
        'reference_customer': f"FELANM-{datetime.now().strftime('%Y-%m-%d')}-
{order.id}"
    }

    task = create_task_with_metadata(task_data, customer_id, creation_method='extern-
al')

    # Lägg till information
    add_task_information(task.id, 'text', report_data['description'], False, custom-
er_id)

    for image_url in report_data.get('images', []):
        add_task_information(task.id, 'image', image_url, False, customer_id)

    # Sätt status
    update_task_status(task.id, 'active')
    update_task_execution_status(task.id, 'not_planned', customer_id)

    # Skicka bekräftelse till kund
```

```
    send_customer_confirmation(customer_id, task.id, report_data.get('contact_phone'))

    return task
```

Funktion: Tidslogg och Historik

```
def log_task_time_change(task_id, event_type, user_id, details=None):
    """
    Logga tidsrelaterad ändring i uppgift.

    Parametrar:
    - task_id: Uppgiftens ID
    - event_type: Typ av händelse (t.ex. 'created', 'planned_rough',
'status_change_on_site')
    - user_id: Användaren som gjorde ändringen
    - details: Extra detaljer (optional)
    """
    log_entry = {
        'timestamp': datetime.now().isoformat(),
        'event_type': event_type,
        'user_id': user_id,
        'details': details
    }

    task = get_task(task_id)
    time_log = task.time_log or []
    time_log.append(log_entry)

    query = "UPDATE tasks SET time_log = %s WHERE id = %s"
    execute_query(query, [json.dumps(time_log), task_id])

def get_task_time_history(task_id):
    """
    Hämta fullständig tidshistorik för uppgift.

    Returnerar: Lista av logg-händelser sorterat efter tid
    """
    task = get_task(task_id)
    return sorted(task.time_log, key=lambda x: x['timestamp'])
```

3.2.4 Hur ska det integreras med befintligt system?

Integration A: Orderkoncept och Uppgiftsskapande

När orderkoncept söker sig genom kluster:

1. För varje matchat objekt: Skapa uppgift med `create_task_with_metadata()`
2. Populera uppgift med objektets metadata (plats, kontakter, etc.)
3. Hantera strukturartiklar: Skapa alla beroendeuppgifter
4. Sätt önskade tidsfönster baserat på kundens preferenser

Integration B: Snöret (Planering)

När uppgifter planeras i “Snöret”:

1. Hämta uppgiftens önskade tidsfönster
2. Matcha mot utförarens tillgänglighet
3. Optimera resvägar baserat på GPS-koordinater
4. Kontrollera beroendelogik innan schemaläggning
5. Uppdatera `execution_status` till ‘planned_fine’
6. Logga planeringen i `time_log`

Integration C: Utförare-app

När utförare utför uppgifter:
1. Visa uppgiftens fullständiga metadata (plats med What3Words, instruktioner, bilder)
2. Tillåt utförare att uppdatera uppgift (status, bilder, anteckningar)
3. Uppdatera `execution_status` steg för steg (on_way -> on_site -> completed)
4. Logga alla ändringar med `log_task_time_change()`
5. Kontrollera beroendelogik: Notifiera system om beroende uppgifter kan påbörjas

Integration D: Fakturering

När uppgift ska faktureras:
1. Kontrollera `billing_model` för uppgiften
2. Om 'per_task': Fakturera direkt
3. Om 'per_order': Vänta tills alla uppgifter i ordern är slutförda
4. Om 'monthly': Samla alla uppgifter för månaden
5. Använd referenser (`reference_object` , `reference_order` , `reference_customer`) på fakturan
6. Uppdatera `execution_status` till 'invoiced'

Integration E: Kundportal

Kundens webb-/mobilgränssnitt:
1. Visa kundens uppgifter (filtrerat på `customer_id`)
2. Visa statusuppdateringar i realtid
3. Tillåt kund att skapa felanmälningar via `create_task_from_external_report()`
4. Visa bildgalleri från uppgifter (TaskInformation med type='image')

3.2.5 Prioritet

KRITISK - Detta är kärnan i uppgiftshanteringen

3.3 Utökad Informationsvy (Utförare/Team)

3.3.1 Vad behöver byggas?

Användargränssnitt

Ny vy: "Uppgifter och information"
- Specifik vy för utförare/team
- Visar dagens/veckans uppgifter
- All relevant metadata per uppgift
- Optimerad för mobil (utförare använder ofta telefon/platta)

3.3.2 Vilka datamodeller behöver skapas/uppdateras?

Inga nya datamodeller, använder befintliga `Task` och relaterade modeller.

3.3.3 Vilka funktioner behöver implementeras?

Funktion: Hämta Utförares Uppgifter

```
def get_performer_tasks(performer_id, start_date=None, end_date=None,
                        include_metadata=True):
    """
    Hämta alla uppgifter för en utförare/team.

    Parametrar:
    - performer_id: Utförare eller team ID
    - start_date: Från datum (optional)
    - end_date: Till datum (optional)
    - include_metadata: Om all metadata ska inkluderas (standard: True)

    Returnerar: Lista av Task objekt med all metadata
    """
    query = """
        SELECT t.*
        FROM tasks t
        WHERE (t.assigned_team = %s OR t.assigned_coordinator = %s)
          AND t.execution_status NOT IN ('archived', 'invoiced')
    """
    params = [performer_id, performer_id]

    if start_date:
        query += " AND t.planned_delivery_date >= %s"
        params.append(start_date)

    if end_date:
        query += " AND t.planned_delivery_date <= %s"
        params.append(end_date)

    query += " ORDER BY t.planned_delivery_date, t.start_time"

    tasks = execute_query(query, params)

    if include_metadata:
        for task in tasks:
            # Lägg till full metadata
            task.object = get_object(task.object_id)
            task.desired_timewindows = get_desired_timewindows(task.id)
            task.dependencies = get_task_dependencies(task.id)
            task.information = get_task_information(task.id)

    return tasks
```

Funktion: Generera Dag/Vecka-Vy för Utförare

```
def generate_performer_schedule_view(performer_id, date):
    """
    Generera en strukturerad vy av utförarens schema för given dag.

    Inkluderar:
    - Uppgifter i tidsordning
    - Resvägar mellan uppgifter
    - Beräknad restid
    - All relevant metadata per uppgift
    - OBS-meddelanden och viktiga noteringar

    Returnerar: Strukturerad vy-objekt
    """
    tasks = get_performer_tasks(performer_id, start_date=date, end_date=date)

    schedule = {
        'date': date,
        'performer_id': performer_id,
        'items': []
    }

    previous_location = get_performer_start_location(performer_id, date)

    for task in tasks:
        # Beräkna resväg från föregående plats
        if previous_location:
            travel = calculate_travel(previous_location, task.gps_lat, task.gps_lng)
            schedule['items'].append({
                'type': 'travel',
                'from': previous_location,
                'to': {'lat': task.gps_lat, 'lng': task.gps_lng, 'what3words': task.wh
at3words},
                'distance_km': travel['distance'],
                'duration_hours': travel['duration']
            })

        # Lägg till uppgift
        schedule['items'].append({
            'type': 'task',
            'task': task,
            'start_time': task.start_time,
            'end_time': task.end_time,
            'duration_hours': task.execution_time_hours,
            'object_name': task.object.name,
            'address': task.address,
            'gps': {'lat': task.gps_lat, 'lng': task.gps_lng},
            'what3words': task.what3words,
            'customer_contact': get_object_primary_contact(task.object_id),
            'instructions': get_task_text_information(task.id),
            'images': get_task_images(task.id),
            'important_notes': extract_important_notes(task)
        })

        previous_location = {'lat': task.gps_lat, 'lng': task.gps_lng}

    # Beräkna statistik för dagen
    schedule['statistics'] = calculate_day_statistics(schedule['items'])

    return schedule
```

3.3.4 Hur ska det integreras med befintligt system?

- Lägg till ny vy i utförare-appen
- Använd befintlig uppgiftsdata men presentera det fokuserat för utförare
- Integration med kart-API för resvägar
- Push-notifikationer vid uppdateringar

3.3.5 Prioritet

HÖG - Viktigt för utförare men kan implementeras efter core metadata-funktionalitet

---

DEL 4: KONKRETA EXEMPEL FRÅN KINABS VERKSAMHET

4.1 Scenario 1: Ny BRF med Matavfallshantering

Bakgrund

BRF Rosenlund är en ny kund med 3 byggnader och totalt 45 matavfallsbehållare. De vill ha veckovis tömning av alla behållare.

Implementering med Nya Funktioner

Steg 1: Skapa Kluster med Metadata

```
OBJEKT: BRF Rosenlund (Organisatoriskt)
  Nummer: #ORG-001
  Kund: BRF Rosenlund AB (Org.nr: 556123-4567)
  Fakturamottagare: BRF Rosenlund AB
  Kontakt: Johan Svensson (070-123 45 67, johan@rosenlund.se)
  Prislista: Standard BRF 2025
  Rabattbrev: Volymrabatt 5%
  Tillgänglighet: Mån-Fre 07:00-16:00
  [Metadata: own_inheritable, security: free]

  └─> OBJEKT: Byggnad A (Organisatoriskt)
        Nummer: #BYGG-001-A
        Adress: Storgatan 12A, 12345 Stockholm
        [Ärver: Kund, Kontakt, Prislista, Rabatt, Tillgänglighet]
        [Metadata: own_inheritable, security: free]

        └─> OBJEKT: Port 1 (Organisatoriskt)
              Nummer: #PORT-001-A-1
              Portkod: 1234
              [Ärver: Alla från Byggnad A]
              [Metadata: own_locked, security: locked]

              └─> OBJEKT: Matavfallsbehållare 1 (Fysiskt)
                    Nummer: #23432143423
                    Namn: Matavfallsbehållare Storgatan 12A Port 1
                    GPS: 59.3293, 18.0686
                    What3Words: ///exempel.adress.här
                    Storlek: 240L
                    Märke: Kärl & Co AB
                    Serienummer: KC-240-8832
                    Objekttyp för artiklar: "Matavfall"
                    Tillgängliga datum: Mån, Ons, Fre (ärvt + specificerat)
                    [Ärver: Kund, Kontakt, Prislista, Rabatt, Tillgänglighet, Adress]
                    [Egen metadata: Storlek, Märke, Serienummer - own_locked, security: l
ocked]

                    Bildgalleri:
                      - 2025-01-04: Foto vid installation
```

Steg 2: Skapa Orderkoncept för Veckovis Tömning

```
ORDERKONCEPT: "Veckovis tömning BRF Rosenlund"
  Kund: BRF Rosenlund AB
  Prislista: Standard BRF 2025

  ARTIKEL: "Tömning matavfall 240L"
    Artikelnummer: ART-100
    Objekttyp filter: "Matavfall"
    Återkommande: Veckovis
    Önskad leverans:
      Alt 1: Varje vecka, Måndag, 08:00-12:00
      Alt 2: Varje vecka, Onsdag, 13:00-16:00
```

Steg 3: Orderkoncept Söker och Skapar Uppgifter

Systemet hittar alla 45 matavfallsbehållare och skapar uppgifter:

```
UPPGIFT #4523: Tömning matavfall - #23432143423
  Objekt: Matavfallsbehållare #23432143423
  Artikel: ART-100
  Order: ORDER-2025-0234

  Skapat av: System (Automatik)
  Skapningssätt: Automatik (Återkommande order)
  Koppling: 2501041000

  Önskade tidsfönster:
    1. Vecka 15, Måndag, 08:00-12:00 (prioritet 1)
    2. Vecka 15, Onsdag, 13:00-16:00 (prioritet 2)

  Plats:
    Adress: Storgatan 12A, 12345 Stockholm
    GPS: 59.3293, 18.0686
    What3Words: ///exempel.adress.här

  Kund: BRF Rosenlund AB
  Kontakt: Johan Svensson (070-123 45 67)

  Instruktioner: "OBS! Smalt infart, använd mindre fordon. Portkod: 1234"

  Fakturering: Månadsfaktura
  Referens: BRF Rosenlund - Matavfall veckovis

  Status: Active
  Utförande-status: Ej planerat
```

(Detta upprepas för alla 45 behållare)

Steg 4: Planering (Snöret)

Planerare ser alla 45 uppgifter och planerar in dem:
- Grupperar uppgifter geografiskt (Byggnad A, B, C)
- Väljer Måndag 08:00-12:00 (prioritet 1)
- Tilldelar Team B (Johan + Erik)
- Optimerar resväg baserat på GPS-koordinater
- Uppdaterar status: Planerat grovt → Planerat fint

Steg 5: Utförande

Team B ser i sin app:

```
Måndag 2025-04-07
═══════════════════════════════════════
07:00  Start från Kinab (Järna)

08:00-08:12  RESA: Kinab → Storgatan 12A
             12 km, 12 min

08:12  UPPGIFT: Tömning #23432143423
       BRF Rosenlund - Matavfallsbehållare Storgatan 12A Port 1
       📍 ///exempel.adress.här
       ⚠️ OBS! Smalt infart, mindre fordon. Portkod: 1234
       👤 Johan Svensson (070-123 45 67)
       ⏱️ 30 min

       [Starta] [Kontakt] [Karta]

08:42  UPPGIFT: Tömning #23432143424
       BRF Rosenlund - Matavfallsbehållare Storgatan 12A Port 2
       ...
```

Team B uppdaterar status:
- 08:00: På väg
- 08:12: På plats
- 08:42: Utfört

Steg 6: Felanmälan från Kund

Johan Svensson (kontakt hos BRF) loggar in på kundportalen och rapporterar:

```
"Lock på behållare #23432143423 sitter hårt och går inte att öppna."
+ 2 bilder bifogade
```

Systemet skapar automatiskt:

```
ORDER: ORDER-2025-0589 (Extern felanmälan)
  Kund: BRF Rosenlund AB
  Skapat: 2025-03-10 14:23 via kundportal

UPPGIFT #4891: Underhåll - Lock sitter hårt
  Objekt: #23432143423
  Artikel: ART-500 (Underhåll behållare)

  Skapat av: Johan Svensson (Kund)
  Skapningssätt: Extern inmatning (felanmälan)

  Plats: Storgatan 12A (GPS + What3Words från objekt)

  Information:
    Text: "Lock på behållare #23432143423 sitter hårt och går inte att öppna."
    Bilder: [bild1.jpg, bild2.jpg]

  Önskad leverans: "Så snart som möjligt"

  Status: Active
  Utförande-status: Ej planerat

  Tilldelning: Ej tilldelad än
```

System skickar SMS till Johan: “Tack för din felanmälan. Vi har mottagit den och kommer åtgärda problemet inom 2 arbetsdagar. Referens: FELANM-2025-0123”

Planerare ser felanmälan, tilldelar till Servicetekniker Lisa, planerar till nästa dag.

Steg 7: Fakturering

I slutet av månaden:
- Systemet samlar alla uppgifter för BRF Rosenlund med billing_model=’monthly’
- Skapar faktura:
```
BRF ROSENLUND AB
Faktura 2025-04

Veckovis tömning matavfall:
- 45 behållare x 4 veckor = 180 tömningar
- Pris per tömning: 150 kr
- Subtotal: 27 000 kr
- Rabatt 5%: -1 350 kr
- Total: 25 650 kr

Extra service:
- Underhåll behållare (FELANM-2025-0123): 800 kr

TOTALT: 26 450 kr
```
- Uppdaterar alla uppgifter: execution_status → ‘invoiced’

4.2 Scenario 2: Strukturartikel - Byte av Container

Bakgrund

Fast AB Eken behöver byta ut sin stora container (Objekt #98765). Detta kräver flera steg i rätt ordning.

Implementering med Beroendelogik

Steg 1: Orderkoncept med Strukturartikel

```
ORDERKONCEPT: "Byte av container"
  Kund: Fast AB Eken
  Objekt: Container #98765

  STRUKTURARTIKEL: "Byte av container 20m³"
    Artikel: STR-200

    Delartiklar (med beroendelogik):
      1. ART-201: Föravisering (telefon)
      2. ART-202: Hämtning gammal container (beroende av #1)
      3. ART-203: Leverans ny container (beroende av #2)
      4. ART-204: Efteravisering (SMS) (beroende av #3)
```

Steg 2: Automatiskt Skapade Beroendeuppgifter

Systemet skapar automatiskt alla 4 uppgifter:

```
UPPGIFT #5001: Föravisering - Byte container #98765
  Objekt: Container #98765
  Artikel: ART-201
  Order: ORDER-2025-0678

  Uppgiftstyp: Avisering
  Utförandetid: 0.1h (6 min telefonsamtal)

  Beroendelogik:
    - Ingår i strukturartikel STR-200
    - Måste göras först (ingen beroende uppgift)

  Information:
    Text: "Ring kund och bekräfta tid för byte av container"
    Kontakt: Anders Andersson (Fast AB Eken, 070-987 65 43)

  Status: Active
  Utförande-status: Ej planerat

  ═══════════════════════════════════════

UPPGIFT #5002: Hämtning gammal container #98765
  Objekt: Container #98765
  Artikel: ART-202
  Order: ORDER-2025-0678

  Uppgiftstyp: Containerhantering
  Utförandetid: 0.7h
  Restid: 0.3h

  Plats:
    Adress: Industrivägen 45, 12345 Stockholm
    GPS: 59.1234, 18.5678
    What3Words: ///container.hämta.här

  Beroendelogik:
    - Ingår i strukturartikel STR-200
    - Beroende av: Uppgift #5001 (Föravisering) måste vara slutförd

  Status: Active
  Utförande-status: Ej planerat (BLOCKERAD av #5001)

  ═══════════════════════════════════════

UPPGIFT #5003: Leverans ny container
  Objekt: Container #98765 (ny)
  Artikel: ART-203
  Order: ORDER-2025-0678

  Uppgiftstyp: Containerhantering
  Utförandetid: 0.5h
  Restid: 0h (samma plats som #5002)

  Plats: [Samma som #5002]

  Beroendelogik:
    - Ingår i strukturartikel STR-200
    - Beroende av: Uppgift #5002 (Hämtning) måste vara slutförd

  Status: Active
  Utförande-status: Ej planerat (BLOCKERAD av #5002)
```

```
  ═══════════════════════════════════

UPPGIFT #5004: Efteravisering - Byte slutfört
  Objekt: Container #98765
  Artikel: ART-204
  Order: ORDER-2025-0678

  Uppgiftstyp: Avisering
  Utförandetid: 0.05h (SMS)

  Beroendelogik:
    - Ingår i strukturartikel STR-200
    - Beroende av: Uppgift #5003 (Leverans) måste vara slutförd

  Information:
    Text: "Skicka SMS till kund: 'Byte av container slutfört. Nya container-nummer:
#99999'"

  Status: Active
  Utförande-status: Ej planerat (BLOCKERAD av #5003)
```

Steg 3: Planering med Beroendelogik

Planerare ser uppgifterna:
- **Uppgift #5001 (Föravisering)**: Planeras till Tisdag 10:00
- **Uppgift #5002, #5003, #5004**: Visas som “BLOCKERAD - Väntar på beroende uppgifter”

Steg 4: Utförande Steg-för-Steg

Tisdag 10:00 - Föravisering

```
Administrator ringer Anders Andersson:
  "Hej! Vi kommer på Onsdag 08:00 för att byta er container. Passar det?"
  Anders: "Ja, det passar utmärkt!"

Administrator markerar Uppgift #5001 som UTFÖRD
  → System notifierar automatiskt: "Uppgift #5002 kan nu planeras"
```

Efter Föravisering Slutförd

```
Planerare ser nu:
  ✅ Uppgift #5001: Föravisering - UTFÖRD
  🟡 Uppgift #5002: Hämtning - KAN NU PLANERAS
  🔒 Uppgift #5003: Leverans - Blockerad (väntar på #5002)
  🔒 Uppgift #5004: Efteravisering - Blockerad (väntar på #5003)

Planerare planerar #5002 till Onsdag 08:00, tilldelar Team C
```

Onsdag 08:00 - Hämtning

```
Team C:
  08:00 - På väg till Industrivägen 45
  08:15 - På plats
  08:45 - Container hämtad, utförd

System: Uppgift #5002 markerad som UTFÖRD
  → Automatisk notifiering: "Uppgift #5003 kan nu utföras"
```

Onsdag 08:45 - Leverans (Omedelbart Efter)

```
Team C fortsätter direkt (samma plats):
  08:45 - Börjar leverans av ny container
  09:15 - Ny container placerad, utförd

System: Uppgift #5003 markerad som UTFÖRD
  → Automatisk notifiering: "Uppgift #5004 kan nu utföras"
```

Onsdag 09:15 - Efteravisering (Automatisk)

```
System skickar automatiskt SMS till Anders Andersson:
  "Byte av container slutfört. Nya container-nummer: #99999. Tack för att ni använder
Kinab!"

System: Uppgift #5004 markerad som UTFÖRD automatiskt
```

Steg 5: Resultat

Alla 4 uppgifter slutförda i rätt ordning, automatiskt hanterat av beroendelogiken.

```
ORDER-2025-0678: Byte av container #98765
  Status: Alla uppgifter slutförda

  ✅ #5001: Föravisering (Tisdag 10:00)
  ✅ #5002: Hämtning (Onsdag 08:15)
  ✅ #5003: Leverans (Onsdag 08:45)
  ✅ #5004: Efteravisering (Onsdag 09:15)

  Total tid: 1 dag, 23 timmar, 15 minuter
  Fakturering: Per order (2 500 kr)
```

4.3 Scenario 3: Flera Önskade Tidsfönster

Bakgrund

BRF Lindgren vill ha tömning av matavfall men har begränsad tillgänglighet pga. renoveringsarbete.

Implementering med Flera Tidsfönster

Steg 1: Kunden Anger Preferenser

```
ORDERKONCEPT: "Tömning matavfall BRF Lindgren"
  Kund: BRF Lindgren
  Objekt: Matavfallsbehållare #77777

  Önskade tidsfönster (i prioritetsordning):
    1. Vecka 16, Måndag, 08:00-10:00
    2. Vecka 16, Onsdag, 14:00-16:00
    3. Vecka 17, Tisdag, 07:00-09:00
    4. Vecka 17, Torsdag, 13:00-15:00
```

Steg 2: System Skapar Uppgift med Alla Tidsfönster

```
UPPGIFT #6001: Tömning matavfall BRF Lindgren
  Objekt: #77777

  Önskade tidsfönster:
    1. Vecka 16, Måndag, 08:00-10:00 (prioritet 1)
    2. Vecka 16, Onsdag, 14:00-16:00 (prioritet 2)
    3. Vecka 17, Tisdag, 07:00-09:00 (prioritet 3)
    4. Vecka 17, Torsdag, 13:00-15:00 (prioritet 4)
```

Steg 3: Planering med Optimering

Planeraren ser uppgiften och kan nu:

1. Se alla möjliga tidsfönster
2. Välja det som passar bäst i schemat
3. Optimera mot andra uppgifter i närområdet

```
Planerare:
  "Vecka 16 Måndag 08:00-10:00 är fullt med andra uppdrag.
   Vecka 16 Onsdag 14:00-16:00 passar perfekt - vi har 3 andra uppdrag i närområdet
då!"

  → Väljer tidsfönster #2 (Vecka 16, Onsdag, 14:00-16:00)
  → Planerar uppgiften
```

Steg 4: Automatisk Bekräftelse till Kund

```
System skickar SMS till BRF Lindgren:
  "Din tömning av matavfall är planerad till Onsdag 2025-04-16 kl 14:00-16:00.
   Du angav detta som alternativ 2. Referens: ORDER-2025-0789"
```

Fördel: Flexibilitet för både kund och Kinab, optimal planering

---

4.4 Scenario 4: What3Words för Exakt Platsbestämning

Bakgrund

Stort industri område med flera identiska byggnader. Traditionella adresser är otydliga (“Industrivägen 12” har 8 olika ingångar).

Implementering med What3Words

Steg 1: Objekt Definieras med What3Words

```
OBJEKT: Container #88888
  Namn: Container Industrivägen 12 - Nordöstra hörnet
  Adress: Industrivägen 12, 12345 Stockholm
  GPS: 59.1111, 18.2222
  What3Words: ///industriområde.nordöst.container

  Instruktioner: "What3Words är exakt plats för containern.
                  Går INTE att hitta med bara gatuadress."
```

Steg 2: Uppgift Skapas med What3Words

```
UPPGIFT #7001: Tömning container #88888
  Plats:
    Adress: Industrivägen 12, 12345 Stockholm (generell)
    GPS: 59.1111, 18.2222
    What3Words: ///industriområde.nordöst.container (EXAKT)
```

Steg 3: Utförare Använder What3Words

Team D ser i sin app:

```
UPPGIFT: Tömning container #88888
📍 EXAKT PLATS: ///industriområde.nordöst.container

[Öppna i What3Words App] [Navigera med Google Maps]

⚠️ OBS: Använd What3Words för att hitta exakt plats.
         Området har många identiska byggnader.
```

Team D klickar “Navigera med Google Maps”:
- What3Words konverteras automatiskt till GPS
- Google Maps navigerar till exakt 3x3m ruta
- Team hittar direkt rätt container

Resultat: Ingen tid bortkastad på att leta, perfekt första gången

---

4.5 Scenario 5: Metadata-arv och Säkerhet

Bakgrund

BRF Storhuset har olika prislistor för olika byggnader (gamla avtalet vs nya avtal). Vissa data ska inte ändras.

Implementering med Arvslogik och Säkerhet

Steg 1: Klusterstruktur med Olika Priser

```
BRF STORHUSET (Organisatoriskt)
  Kund: BRF Storhuset AB
  Kontakt: Maria Chef (070-111 22 33)
  Prislista: [EJ SATT PÅ DENNA NIVÅ]
  [Metadata: own_inheritable, security: free]

  ├─> BYGGNAD ÖST (Organisatoriskt)
  │    Prislista: Gammalt Avtal 2023 (pris: 120 kr/tömning)
  │    [Metadata: own_inheritable, security: LOCKED]
  │    "Denna prislista får EJ ändras utan godkännande från VD"
  │
  │    └─> Matavfallsbehållare #11111
  │         Ärver: Prislista Gammalt Avtal 2023 (låst)
  │         Pris per tömning: 120 kr (ärvt, låst)
  │
  └─> BYGGNAD VÄST (Organisatoriskt)
       Prislista: Nytt Avtal 2025 (pris: 150 kr/tömning)
       [Metadata: own_inheritable, security: free]

       └─> Matavfallsbehållare #22222
            Ärver: Prislista Nytt Avtal 2025 (fri)
            Pris per tömning: 150 kr (ärvt, ej låst)
```

Steg 2: Användare Försöker Ändra Pris

```
Användare (Planerare) försöker ändra prislista för Byggnad Öst:
  → System: "NEKAD - Prislista är låst och kan inte ändras.
              Kontakta administratör."

Användare (Planerare) försöker ändra prislista för Byggnad Väst:
  → System: "OK - Prislista uppdaterad till Nytt Avtal 2025."
```

Steg 3: Admin Ändrar Låst Data

```
Användare (Admin/VD) ändrar prislista för Byggnad Öst:
  → System kontrollerar: user_has_admin_rights(user_id) = TRUE
  → System: "OK - Prislista uppdaterad."
  → System loggar: "Admin 'Anna Svensson' ändrade låst prislista för Byggnad Öst
                    från 'Gammalt Avtal 2023' till 'Nytt Avtal 2025'
                    den 2025-04-15 10:30"
```

Fördel:
- Flexibel datahantering med arvslogik
- Skydd mot oavsiktliga ändringar
- Full audit-trail för kritiska ändringar

SAMMANFATTNING

Huvudsakliga Ändringar att Implementera

1. Avancerad Objektmetadata (KRITISK)

- Utöka `objects` tabell med nya fält
- Skapa `object_metadata` tabell för flexibel metadata
- Implementera arvslogik för data i klusterhierarkin
- Skapa säkerhetsmodell (fri/låst/justerbar)
- Bildgalleri-funktionalitet per objekt

2. Avancerad Uppgifts- och Ordermetadata (KRITISK)

- Utöka `tasks` tabell massivt med nya fält
- Skapa `task_desired_timewindows` för flera tidsfönster
- Skapa `task_dependencies` för beroendelogik
- Skapa `task_information` för flexibel information
- Implementera What3Words integration
- Implementera omfattande statushantering (10 statusnivåer)
- Implementera extern inmatning (felanmälan från kund)
- Implementera strukturartiklar med automatiska beroendeuppgifter

3. Utökad Informationsvy (HÖG)

- Skapa ny vy för utförare/team
- Visa uppgifter med all relevant metadata
- Optimerad för mobil

Teknisk Komplexitet

Dessa ändringar representerar en **betydande utökning** av systemets komplexitet:
- Många nya databas-tabeller och fält
- Komplex affärslogik (arvslogik, beroendelogik, säkerhet)
- Externa integrationer (What3Words, GPS, felanmälan från kund)
- Omfattande metadata-hantering

Affärsvärde

Ändringarna ger **mycket stort affärsvärde**:
- Fullständig spårbarhet och transparens
- Flexibilitet för olika kunder och situationer
- Automatisering av repetitiva processer
- Bättre kundnöjdhet (flera tidsfönster, felanmälan, transparens)
- Effektivare planering (GPS, What3Words, beroendelogik)
- Säkrare datahantering

Implementationsordning (Rekommendation)

1. **Fas 1** (Kritisk): Avancerad Objektmetadata (2-3 veckor)

2. **Fas 2** (Kritisk): Avancerad Uppgifts- och Ordermetadata - Grundfunktionalitet (3-4 veckor)
3. **Fas 3** (Hög): Beroendelogik och Strukturartiklar (2 veckor)
4. **Fas 4** (Hög): Externa Integrationer (What3Words, Felanmälan) (1-2 veckor)
5. **Fas 5** (Medel): Utökad Informationsvy för Utförare (1 vecka)

Total uppskattad tid: 9-12 veckor

Framtagen för Replit 3.0 Implementation - 2025-01-04